

Лабораторная работа №8

**Реализация основных моделей в дискретно-событийном
подходе**

Улитина Мария Максимовна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Дискретно-событийное моделирование	7
3.2	Модель SIR	7
4	Выполнение лабораторной работы	9
4.1	1. Структура проекта	9
4.2	2. Реализация модели SIR	9
4.3	3. Анализ чувствительности к параметрам (задание 8.5.1)	10
4.4	4. Детерминированная vs стохастическая длительность болезни	11
4.5	5. Оценка производительности (задание 8.5.3)	12
4.6	6. Сохранение результатов в CSV (задание 8.5.4)	12
4.7	7. Вакцинация (задание 8.5.6)	12
4.8	8. Модель SEIR (задание 8.5.7)	13
4.9	9. Результаты моделирования	13
4.10	9. Создание литературного кода	14
5	Выводы	16
	Список литературы	18

Список иллюстраций

4.1	Структура проекта	9
4.2	Функция out	9
4.3	Запуск скрипта	10
4.4	Анализ чувствительности	10
4.5	Детерминированная версия	11
4.6	Стохастическая версия	11
4.7	Бенчмаркинг	12
4.8	Сохранение CSV	12
4.9	Вакцинация	12
4.10	Модель SEIR	13
4.11	Динамика SIR модели	14
4.12	Литературный код	15

Список таблиц

1 Цель работы

Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR. Реализовать стохастическую дискретно-событийную модель в виде программного комплекса на языке Julia. Провести анализ влияния параметров, сравнить со стохастической и детерминированной версиями, оценить производительность и модифицировать модель.

2 Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Провести анализ чувствительности к параметрам (β , c , γ).
5. Реализовать детерминированную версию длительности болезни.
6. Оценить производительность модели с помощью бенчмаркинга.
7. Реализовать сохранение результатов в CSV.
8. Добавить демографические события (рождение и смерть).
9. Реализовать вакцинацию.
10. Расширить модель до SEIR с латентным периодом.

3 Теоретическое введение

3.1 Дискретно-событийное моделирование

Дискретно-событийное моделирование (ДСМ) — подход, при котором состояние системы изменяется только в дискретные моменты времени, соответствующие наступлению событий. В отличие от непрерывного (дифференциальные уравнения) или пошагового (фиксированный шаг по времени) подходов, ДСМ позволяет эффективно моделировать системы, где значимые изменения происходят относительно редко.

В данной работе используется пакет `ConcurrentSim.jl` для языка Julia. Этот пакет предоставляет: - корутины (на основе макроса `@yield`) для описания процессов; - очередь событий с виртуальным временем; - механизмы синхронизации и планирования.

3.2 Модель SIR

Модель SIR описывает распространение инфекции в популяции, разделённой на три группы:

Группа	Обозначение	Описание
S (Susceptible)	Восприимчивые	Здоровые индивиды, которые могут заразиться

Группа	Обозначение	Описание
I (Infected)	Инфицированные	Заразные индивиды, способные передавать инфекцию
R (Recovered)	Выздоровевшие	Индивиды с иммунитетом

Переходы определяются параметрами: - β — вероятность передачи инфекции при контакте (от 0 до 1); - c — среднее число контактов в единицу времени; - γ — скорость выздоровления (величина, обратная средней длительности болезни).

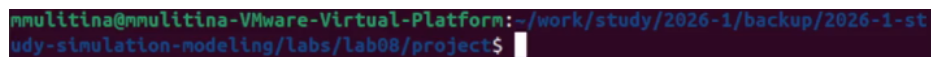
Эффективный коэффициент передачи: $R_0 = \beta \cdot c / \gamma$ — базовое репродуктивное число.

4 Выполнение лабораторной работы

4.1 1. Структура проекта

Работа выполнялась в каталоге проекта:

(рис. 4.1)



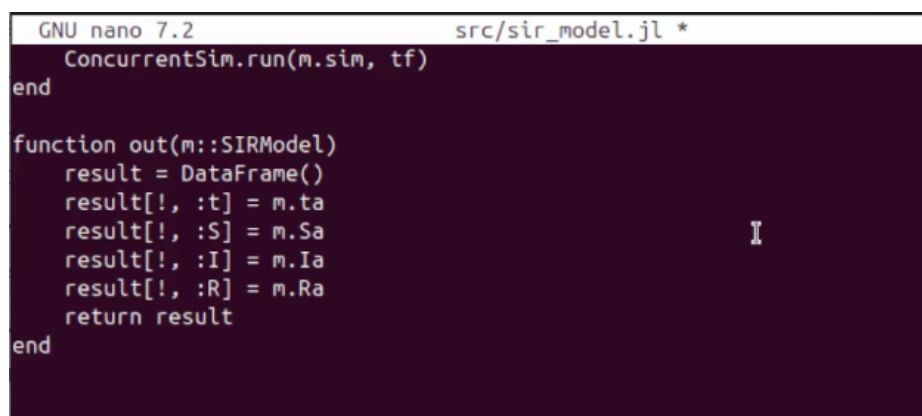
```
mmulitina@mmulitina-VMware-Virtual-Platform:~/work/study/2026-1/backup/2026-1-study-simulation-modeling/labs/lab08/project$
```

Рисунок 4.1: Структура проекта

Рисунок 1 — Рабочая директория проекта

4.2 2. Реализация модели SIR

Основной код модели находился в файле `src/sir_model.jl`. Функция `out(m::SIRModel)` формирует результирующий `DataFrame`: (рис. 4.2)



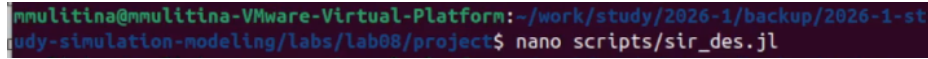
```
GNU nano 7.2 src/sir_model.jl *
ConcurrentSim.run(m.sim, tf)
end

function out(m::SIRModel)
    result = DataFrame()
    result[:, :t] = m.ta
    result[:, :S] = m.Sa
    result[:, :I] = m.Ia
    result[:, :R] = m.Ra
    return result
end
```

Рисунок 4.2: Функция out

Рисунок 2 — Функция вывода результатов

Запуск экспериментов осуществлялся из скрипта `scripts/sir_des.jl`:
(рис. 4.3)



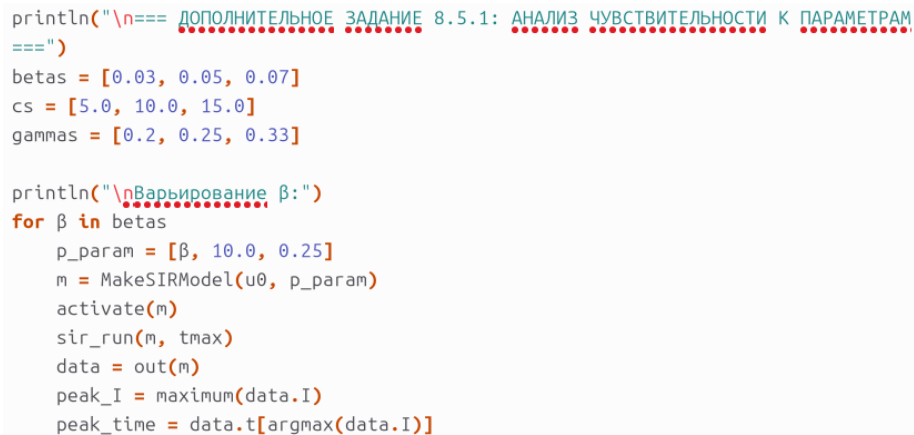
```
mmulitina@mmulitina-VMware-Virtual-Platform:~/work/study/2026-1/backup/2026-1-study-simulation-modeling/labs/lab08/project$ nano scripts/sir_des.jl
```

Рисунок 4.3: Запуск скрипта

Рисунок 3 — Скрипт запуска модели

4.3 3. Анализ чувствительности к параметрам (задание 8.5.1)

Проведён анализ вариации коэффициента передачи β : (рис. 4.4)



```
println("\n=== ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ 8.5.1: АНАЛИЗ ЧУВСТВИТЕЛЬНОСТИ К ПАРАМЕТРАМ ===")
betas = [0.03, 0.05, 0.07]
cs = [5.0, 10.0, 15.0]
gammas = [0.2, 0.25, 0.33]

println("\nВарьирование  $\beta$ :")
for  $\beta$  in betas
    p_param = [ $\beta$ , 10.0, 0.25]
    m = MakeSIRModel(u0, p_param)
    activate(m)
    sir_run(m, tmax)
    data = out(m)
    peak_I = maximum(data.I)
    peak_time = data.t[argmax(data.I)]
end
```

Рисунок 4.4: Анализ чувствительности

Рисунок 4 — Код для анализа чувствительности к параметру β

Результаты анализа:

Параметр	Изменение	Влияние на динамику
$\beta \uparrow$	Увеличение вероятности заражения	Более ранний и высокий пик I
$c \uparrow$	Увеличение числа контактов	Аналогично увеличению β
$\gamma \uparrow$	Более быстрое выздоровление	Снижение пика I

4.4 4. Детерминированная vs стохастическая длительность болезни

Детерминированная версия (фиксированное время выздоровления): (рис. 4.5)

```
if individual.status == :I
  # Детерминированная или стохастическая длительность болезни
  if m.γ > 0
    @yield timeout(env, 1/m.γ) # детерминированная версия
```

Рисунок 4.5: Детерминированная версия

Рисунок 5 — Детерминированная длительность болезни

Для стохастической версии используется экспоненциальное распределение: (рис. 4.6)

```
@yield timeout(env, rand(Exponential(1/μ)))
```

Рисунок 4.6: Стохастическая версия

Рисунок 6 — Стохастическая длительность болезни

4.5 5. Оценка производительности (задание 8.5.3)

С помощью макроса `@benchmark` проведена оценка производительности: (рис. 4.7)

```
benchmark_result = @benchmark sir_run($large_model, $tmax) samples=3
```

Рисунок 4.7: Бенчмаркинг

Рисунок 7 — Бенчмаркинг производительности

4.6 6. Сохранение результатов в CSV (задание 8.5.4)

Реализовано автоматическое сохранение результатов: (рис. 4.8)

```
# Сохранение результатов в CSV
filename = "sir_res.csv"
CSV.write(datadir("sims", filename), data_des)
println("Результаты сохранены в: $(datadir("sims", filename))")
```

Рисунок 4.8: Сохранение CSV

Рисунок 8 — Сохранение результатов в CSV-файл

4.7 7. Вакцинация (задание 8.5.6)

Реализован процесс вакцинации по таймеру: (рис. 4.9)

```
@resumable function vaccinate(env::ConcurrentSim.Simulation, m::SIRModel,
time::Float64, fraction::Float64)
    @yield timeout(env, time)
    N = length(m.allIndividuals)
```

Рисунок 4.9: Вакцинация

Рисунок 9 — Функция вакцинации

4.8 8. Модель SEIR (задание 8.5.7)

Расширение модели до SEIR с латентным периодом: (рис. 4.10)

```
try
  u0_seir = [990, 10, 0] # S, I, R
  p_seir = [0.05, 10.0, 0.25, 0.1] #  $\beta$ ,  $c$ ,  $\gamma$ ,  $\sigma$ 
  seir_model = MakeSIRModel(u0_seir, p_seir, use_seir=true)
  activate(seir_model)
  sir_run(seir_model, tmax)
  data_seir = out(seir_model)

  if nrow(data_seir) > 0 && hasproperty(data_seir, :E)
    @df data_seir plot(
      :t,
      [:S :I :R :E],
      labels = ["S" "I" "R" "E"],
      xlab = "Время",
      ylab = "Численность",
      title = "SEIR модель (с латентным периодом)",
    )
```

Рисунок 4.10: Модель SEIR

Рисунок 10 — Реализация модели SEIR

4.9 9. Результаты моделирования

На основе полученных данных построен график динамики SIR модели. (рис. 4.11)

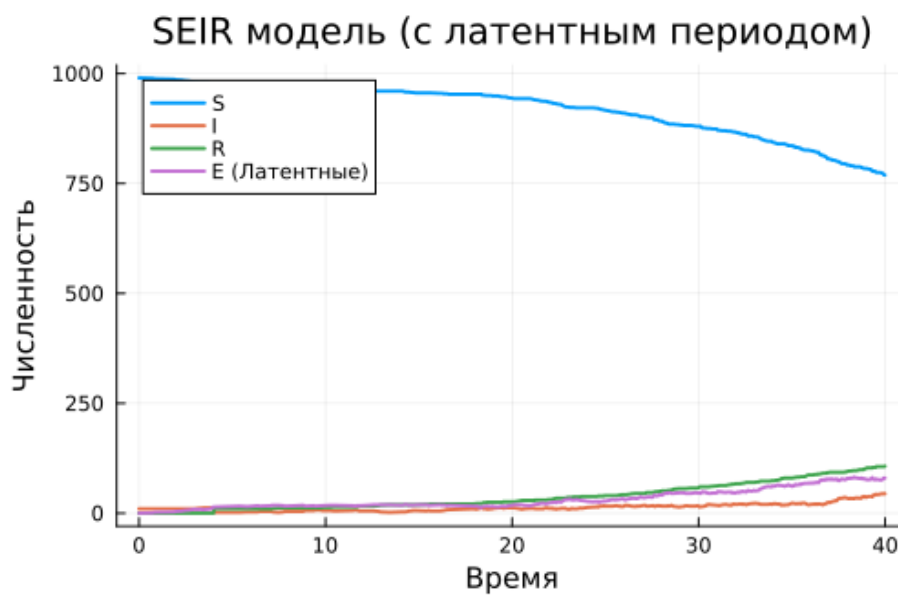


Рисунок 4.11: Динамика SIR модели

Рисунок 11 — Динамика численности групп S , I , R во времени

Сравнение SIR и SEIR:

Характеристика	SIR	SEIR
Пик заболевших	Раньше	Позже
Максимальное I	Выше	Ниже
Скорость распространения	Быстрее	Медленнее
Латентный период	Отсутствует	Присутствует ($E \rightarrow I$)

4.10 9. Создание литературного кода

На основе кода создадим литературный код и сгенерируем ноутбуки (рис. 4.12)



lit.jl

Рисунок 4.12: Литературный код

5 Выводы

В ходе выполнения лабораторной работы №8:

1. **Реализована дискретно-событийная модель SIR** с использованием пакета ConcurrentSim.jl в среде Julia. Модель учитывает индивидуальные траектории индивидов и стохастический характер процессов.
2. **Проведён анализ чувствительности** к параметрам β , c , γ , показавший:
 - Увеличение β и c приводит к более раннему и высокому пику заболевших;
 - Увеличение γ снижает распространение инфекции.
3. **Реализованы обе версии длительности болезни:** детерминированная (фиксированное время) и стохастическая (экспоненциальное распределение).
4. **Выполнена оценка производительности** с помощью бенчмаркинга. Время выполнения модели растёт линейно с увеличением размера популяции.
5. **Реализовано сохранение результатов** в CSV-формат для последующего анализа.
6. **Расширена функциональность модели:**
 - Реализована вакцинация по таймеру;
 - Разработана модель SEIR с латентным периодом.

7. **Дискретно-событийный подход** показал свою эффективность для моделирования эпидемиологических процессов, позволяя естественным образом описывать индивидуальные траектории, стохастические эффекты и сложные сценарии вмешательств.

Список литературы

1. ConcurrentSim.jl Documentation. URL: <https://github.com/pszufe/ConcurrentSim.jl>
2. ResumableFunctions.jl Documentation. URL: <https://github.com/BenLauwens/ResumableFunctions.jl>
3. Banks J., Carson J.S., Nelson B.L., Nicol D.M. Discrete-Event System Simulation. — Pearson, 2014.
4. JuliaLang.org. The Julia Programming Language.
5. DrWatson.jl: Scientific project management in Julia. URL: <https://github.com/JuliaDynamics/DrWatson.jl>